

Predict every shape before you run the cell

Mostly not a coding lab. You write down what you expect, then run it, then mark yourself. Shape errors are the single thing most likely to slow you down between now and December, and this is the cheapest hour you will ever spend on them.

Guided lab

16 minutes

Not graded

Bring a pen

How to do this

1 · build

2 · predict

3 · reach in

4 · reshape

5 · the window

6 · one line

If you finish early

How to do this

**Write first. Run second. Every time.**

The temptation is to run the cell and read the answer. If you do that, this lab teaches you nothing — you will have watched NumPy be correct, which you already believed. The learning happens in the gap between what you predicted and what appeared. Make a two-column page: **my prediction** and **what it did**, and only fill the second column after committing to the first.

Open the provided `lab03.ipynb` and run the import cell. Everything below assumes it.

cell 1

PYTHON

```
import numpy as np
```

TASK 1 · 2 MINUTES

Build four arrays

1

Create each of these, and print `.shape` for every one before you look at the array itself.

- a A 1-D array holding 10, 20, 30, 40 — built from a list.
- b A 2×5 array of zeros.
- c The numbers 0 to 11 in a single row, using `np.arange`.
- d The same twelve numbers arranged as 3 rows by 4 columns.

**One thing that catches everybody in (b)**

`np.zeros(2, 5)` is an error. The shape is a single argument and it is a tuple, so it needs its own brackets: `np.zeros((2, 5))`. You will make this mistake once and then never again.

TASK 2 · 4 MINUTES · THE MAIN TASK

Ten shapes, predicted

2

Type this cell, then **stop**. Write your ten predictions on paper. Then run it.

predict all ten shapes first

PYTHON

```
b = np.arange(12).reshape(3, 4)

print(b.shape)
print(b[0].shape)
print(b[:, 0].shape)
print(b[0, 0].shape)
print(b[0:1].shape)
print(b[1:, 2:].shape)
print(b[:, :].shape)
print(b.reshape(4, 3).shape)
print(b.reshape(12).shape)
print(b.T.shape)
```

(3, 4) · (4,) · (3,) · () · (1, 4) · (2, 2) · (3, 4) · (4, 3) · (12,) · (4, 3)

The two worth dwelling on. `b[0]` gives `(4,)` but `b[0:1]` gives `(1, 4)` — an index removes an axis, a slice keeps it. And `b[0, 0]` gives `()`, because a single number is not an array of one thing; it has no axes at all.

How many did you get? Fewer than seven is completely normal today and would be a problem by session 13, which is exactly why this lab exists now.

TASK 3 · 3 MINUTES

Reach into a grid

3

Using `b` from task 2, write one expression each that produces:

- a the value 7

- b the whole last row

- c the whole second column — `[1, 5, 9]`

- d the bottom-right 2×2 corner

- e every row, but only the first and second columns



Say it out loud as you write it

"Every row, column one." "Rows one onward, columns two onward." If you can say the sentence, the notation follows. If you cannot say it, you are guessing — stop and work out what you actually want first.

TASK 4 · 2 MINUTES

Break reshape on purpose

4

Run each of these and read the error message properly rather than glancing at it.

two of these fail – which, and why?

PYTHON

```
np.arange(12).reshape(2, 6)
np.arange(12).reshape(5, 3)
np.arange(12).reshape(2, 2, 3)
np.arange(12).reshape(3, 5)
```

Write down, in your own words, the single rule that decides whether a reshape is allowed. It is one short sentence about the total number of elements, and knowing it means you will never be surprised by a reshape again.

TASK 5 · 2 MINUTES

The window, demonstrated

5

predict the last line before running

PYTHON

```
A = np.arange(6).reshape(2, 3)
row = A[0]
row[0] = 99

print(A)           # what does A look like now?
```

Then fix it so that changing `row` leaves `A` alone. One method call does it: `A[0].copy()`.



This is Tuesday's trap, one level worse

With lists, `b = a` shared the object. With arrays, even a *slice* shares the memory. When something in session 16 changes that you never assigned to, this is the second thing to check. The first is shapes.

TASK 6 · 3 MINUTES

Tuesday's lab, in one line each

6

Take the grid from Lab 2 and make it an array:

```
g = np.array([[4, 7, 2, 9],
              [1, 8, 6, 3],
              [5, 0, 7, 2]])
```

Now produce each of these in **one line**, and check each answer against what you computed on Tuesday with loops:

- a the total of every number → 54
- b one total per row → [22, 18, 14]
- c one total per column → [10, 15, 15, 14]
- d the largest number anywhere → 9
- e every value with 1 added to it

```
g.sum()           # 54
g.sum(axis=1)     # [22 18 14] - collapse the columns, one number per row
g.sum(axis=0)     # [10 15 15 14] - collapse the rows, one per column
g.max()          # 9
g + 1            # every element, and no loop anywhere
```

Now count the lines. Tuesday's `row_totals` was six lines with two loops and two accumulators; here it is one, and it says what it means. That is the whole argument for NumPy, and you have now made it yourself rather than been told it.

If you finish early

- A Make a 5×5 array where every value is its row index times its column index, without typing the numbers. *Hint:* `np.arange(5).reshape(5, 1) * np.arange(5)` — and if that line surprises you, you have just met Tuesday's topic early.
- B Take `b` from task 2 and produce every value greater than 6, as a 1-D array. *Hint:* try `b > 6` on its own first and look at what it gives you.



Build an 8×8 array that looks like a chessboard — 0 and 1 alternating. There are at least three ways; slicing with a step of 2 is the neat one.